Machine Learning Project

1 Data and Domain Understanding and Exploration

1.1 Meaning and Type of Features; Analysis of Distributions

The Autotrader car dataset has 402,005 observations (rows) and 12 features (columns). It includes records of used cars, featuring attributes relevant to their valuation, such as mileage, brand, model, fuel type, and body type. The main goal is to produce a regression model for predicting the selling price given the characteristics of the cars in the historical data. The features can be classified as follows:

Data type	Features	Characteristic and role
Numerical (Quantitative)	Mileage, Price, year_of_registration	These features are measured as either continuous or discrete quantities (e.g., the number of car seats), and they are used to apply descriptive statistics for detecting outliers and assessing skewness in the data.
Categorical (Qualitative)	Standard_colour, Standard_make, Standard_model, Vehicle_condition, body_type, fuel_type	These features are descriptive attributes used to classify data. These features are stored as object (string) types and need to be encoded into a numerical format before being used in machine learning models.
Categorical (Binary)	crossover_car_and_van	This feature is Boolean or Binary qualitative data, identifying a specific type of vehicle (van or car).
Identifier/code	reg_code, public_reference	This is structural data used for unique referencing. These features can be removed.

Table 1: Data Types and Feature Classifications

The target variable for the prediction is price_winsorized, which denotes the selling price after managing the extreme outliers. This feature is a continuous numerical (quantitative) variable that displays a wide range of prices, from £120 to £9,999,999. The extremely high price acts as an outlier and can distort model training.

So, a winsorized version of the price was created, which cuts extreme values while preserving the overall structure of the data. Price distribution that has a mean of £16,435 and a standard deviation of £14,463, with the median being stable at £12,600. The role and characteristics make the price a central and informative feature for understanding vehicle valuation across different market segments and, most importantly, ensuring stability and robustness for the following modelling stage.

The standard_make feature is a categorical variable that shows the manufacturer or the brand of the cars. This brand feature is one of the most important predictors because the brands hold accumulated value and reputation across the industry. The dataset contains a total of 110 unique manufacturers.

Figure 2 shows the distribution is highly imbalanced and shows there are a few brands that dominate most of the market share. For example, the top five brands, such as BMW (9.30%), Audi (8.78%), Volkswagen (8.52%), Vauxhall (8.38%), and Mercedes-Benz (7.94%), account for 43% of the data dataset.

The large imbalance and substantial number of unique brand categories make it challenging for prediction modelling. The model will be able to predict the top 5 brands quite well, as they have a lot of data. However, there is a risk of error in the remaining 105 categories, as there is little data. The model may overfit or produce inaccurate predictions.

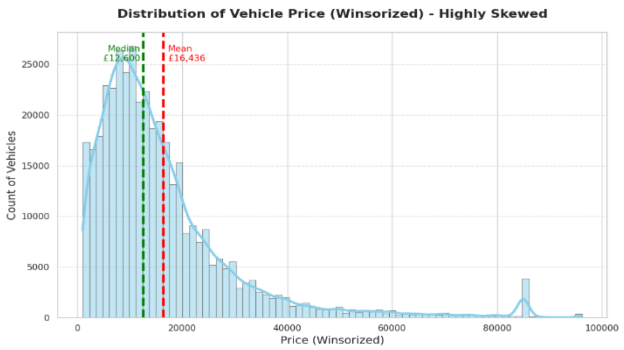

Figure 1: A histogram showing the distribution of vehicle prices winsorised.

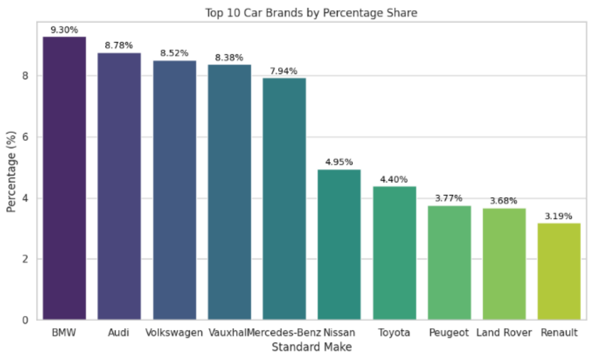

Figure 2: A bar plot showing the top 10 brands by percentage share.

Mileage Distribution and Modelling Implications

Mileage serves as a key indicator of vehicle condition and depreciation patterns. Figure 3 shows the distribution of mileage values capped at the 99^{th} percentile for readability. The histogram uncovers a strongly right-skewed trend, with most cars having low mileage, while fewer sit on the long tail representing high-use cars.

The average mileage (40,000 miles) is notably higher than the median (30,000 miles), confirming that the extreme outliers need to be sorted before the modelling. This skewness matches what we usually see in the used car market behaviour, where newer and barely used cars dominate and show up way more often, whereas extremely high mileage cars occur less frequently in the dataset. Because of these distributional properties, raw mileage may disproportionately bias regression models if left untransformed, making the results not stable and inflating error metrics.

To address this skewness, transformations such as logarithmic scaling and derived features like the vehicle age were used to normalise variance and capture depreciation trends more effectively. This ensures that models learn general usage effects rather than being biased by extreme cases.

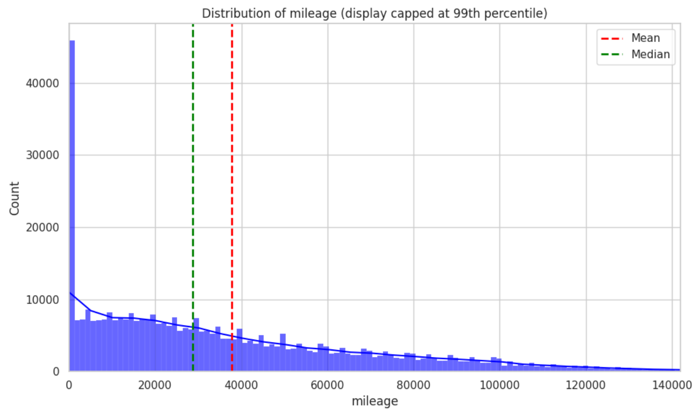

Figure 3: A histogram showing the distribution of vehicle mileage (capped at the 99th percentile) with mean and median indicators.

1.2. Analysis of Predictive Power of Features

Spearman correlation analysis was used to check how well numeric features predict the main target variable. As seen in Figure 4, the nine most correlated ones appear ordered by relationship strength to price. The findings reinforce that wear and usage aspects shape pricing trends more than others. For example, take vehicle age ($\rho=-0.71$) and mileage variants such as log_mileage ($\rho=-0.65$), showing both tied closely but inversely to price. The relationship between Age_x_Mileage ($\rho=-0.70$) shows how age and use work together. However, the year of registration correlated positively ($\rho=+0.71$) but was collinear with Vehicle_age, so only one was kept avoiding multicollinearity.

Other features include condition_encoded ($\rho=+0.32$), showing the price premium for new vehicles, and model_popularity ($\rho=+0.15$), which brings extra insight.

These results shaped preprocessing decisions such as using log transforms on mileage, adjusting scale for distance-focused models, or dropping overlapping variables to boost performance.

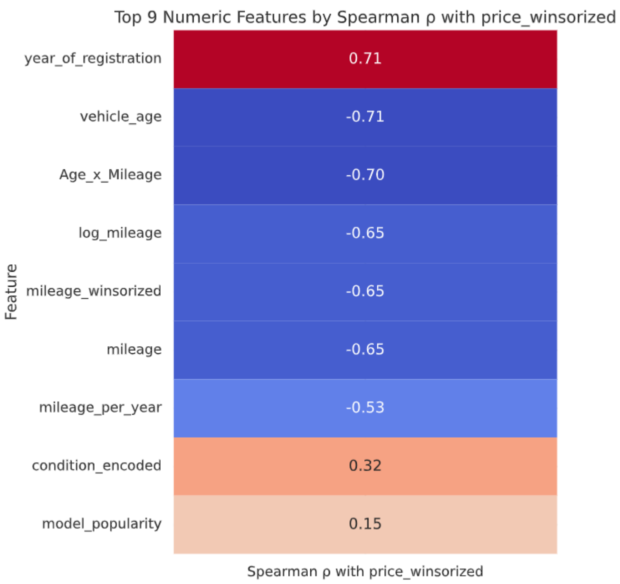

Figure 4. Top numeric features ranked by Spearman correlation with price_winsorized.

The scatter plot (Figure 5) showing the relationship between price and vehicle age reveals how car values change unevenly over time, using a LOWESS smooth curve method that highlights three clear financial phases. At the first stage, from 0 to 25 years, vehicle prices drop fast since parts, design, and tech become outdated. By 30 years, vehicles are at their cheapest, because they are too old to be dependable but not old enough to be vintage; a lot of vehicles are scrapped at this stage.

However, when cars reach 35 years old, their values increase, as most of the cars in this stage are damaged and broken. The ones that survived become rare and vintage, and people pay a premium due to rarity.

The U-shaped relationship emphasised by the red LOWESS curve suggests older cars tend to cost less, but most vehicles are younger than 2 decades, which makes age a key factor in value loss. To keep data balanced, winsorisation was used to cap extreme outliers near £85,000, ensuring scale consistency.

Figure 5: A scatter plot showing price vs vehicle age.

Figure 6 shows SUVs and electric vehicles command higher prices, indicating body type and fuel type notably impact valuation. The boxplot draws attention to the fact that bigger cars like SUVs, MPVs, campers, pickups, and coupes tend to cost more than compact ones such as hatchbacks or saloons. These larger body types often sit at the upper end of the market. In comparison, smaller cars usually fall into the cheaper and affordable price buckets. Expensive cars show broader pricing, suggesting distinct cuts and extras. This range indicates a difference in what each body type offers.

The violin chart for fuel type shows that electric and hybrid cars usually cost more than petrol or diesel ones, due to advanced tech and how they are marketed. Diesel and petrol hold stable in cheaper, middle price brackets, which makes sense since they are widely used and budget friendly. In practice, this means both fuel type and body type are strong predictors when predicting price, and so these features need smart encoding before modelling.

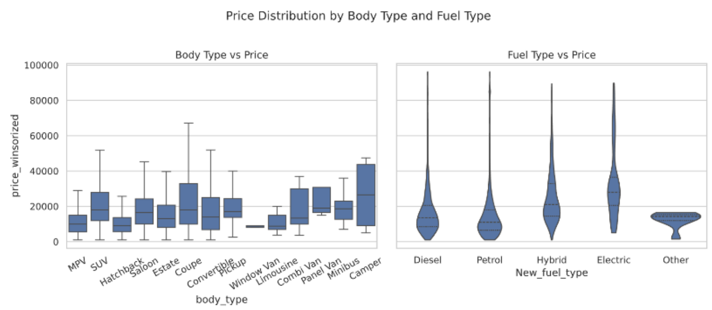

Figure 6: Price distribution across body type and fuel types.

1.3 Data Processing for Data Exploration and Visualisation

Figure 7 shows the distribution of car prices, showing a strong positive skew. Most cars sit between £5000 and £15000, yet some extreme outliers drag the graph wide, like one listed at £9,999,999. Although luxury and high-performance brands like Ferrari, Lamborghini, and McLaren naturally cost more than the average, these extreme anomalies distort visual interpretation and statistical summaries. To deal with this problem, winsorisation was applied at the 1st and 99th percentiles, capping outliers while preserving realistic luxury and high-performance segments. This change ensures charts display the standard market range without losing meaningful variation, improving clarity for exploratory analysis.

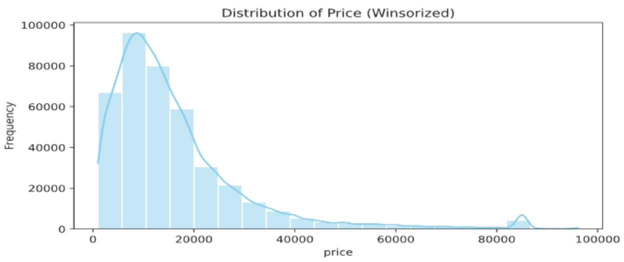

Figure 7: Histogram showing the price column is heavily skewed to the right.

The boxplot and histogram (Figure 8) display the vehicle mileage, how far cars have been driven. In this visual, it is focused on the 0 to 100,000 miles range, so it is easier to see and understand. A substantial proportion of the cars sit under 50,000 miles, though the histogram shows some long tail of higher mileage cars, suggesting a strong right skew. The boxplot confirms this pattern, showing a tight interquartile range and a couple of outliers outside the whiskers.

If the range adjustment has not been made, extreme values, some of which reach up to a million miles, would compress the scales and obscure what most cars look like. Limited to 100,000 miles for a temporary step for EDA, just to make data and insights clearer; the data has not been changed. Instead of hiding the extreme outliers, this approach kept the outliers in the data while making the main patterns easier to see.

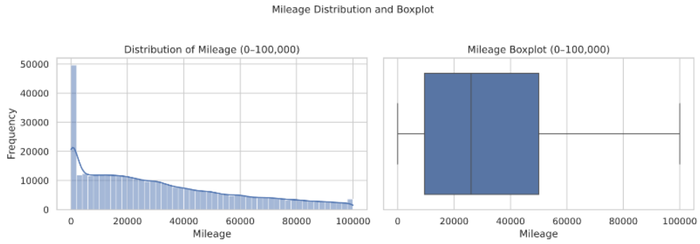

Figure 8: showing the mileage distribution using two visuals (boxplot and histogram).

2. Data Processing for Machine Learning

2.1. Dealing with Missing Values, Outliers, and Noise

There are 33,311 missing values in the year_of_registration feature. To address this large gap while avoiding the significant data loss associated with dropping rows, an imputation approach was implemented. The first step taken was filling the missing year_of_registration based on certain conditions, for example, if the mileage is greater than or equal to 0 and less than or equal to 2000, and the reg_code is missing or empty. The vehicle condition is equal to “NEW”; replace missing values in year_of_registration with 2020. This is done because the dataset is 5 years old and the max year is 2020; thus, any new record on the dataset can have 2020 as the year of registration. However, it improved the data quality.

Another similar logical imputation method was used for inferring missing registration years for “USED” cars from partial reg_code records. First, a new temporary numerical column, reg_code, was created to identify records that can be converted. For the ones that were converted, then the code fills the missing year_of_registration if the reg_code is between 2-9, mapped to year_of_registration 2002-2009 and between 10-19 to years 2010-2019, by adding 2000 to the numeric reg_code. After filling in the missing values, the temporary column was removed to maintain data quality. After implementing these methods, there were still significant missing values for the year_of_registration and reg_code columns. To address this and enhance the accuracy of the incomplete records, a detailed mapping dictionary (reg_to_year) was created, linking UK vehicle registration codes (e.g., 51, 02, 70) to their corresponding year of registration.

Before using the mapping, the year_of_registration column was converted to a nullable integer type to adjust the missing values, and the reg_code column was cleaned by converting it to a string and removing any whitespace. The mapping was then used to fill in the missing year_of_registration entries where the reg_code matched DVLA registration patterns. For example, vehicles with reg_code “09” or “59” were assigned a year_of_registration of 2009, while those with “20” or “70” were assigned 2020. This method was chosen because it aligns the data with actual registration identifiers, ensuring accuracy and consistency for analyses, such as tracking price trends over vehicle age.

To manage missing values in the standard_colour column, a targeted imputation approach was used. First, the average colour for each brand was calculated and applied to fill missing values within that group. If the standard make had no average colour, the most common colour in the dataset was used instead. This imputation technique ensures that missing values are replaced with contextually relevant data, enhancing the consistency and accuracy of the model's predictions and insights.

However, similar imputations were managed for missing values in the mileage column, using a grouped median approach. The dataset was grouped by standard make, standard model and year_of_registration; these are features likely to influence mileage. With each group, the median mileage was calculated and used to fill missing values, and if the group lacked sufficient data, the overall median mileage of the dataset was used as a fallback. Mileage is a crucial factor in pricing and buyer decision-making; if the column is missing values/data, it may be undervalued or excluded from the pricing model. Thus, by imputing missing mileage using the grouped median, maintain pricing accuracy.

These methods improved the completeness, reduced noise, and prepared the dataset for featuring engineering, categories comparison and insights.

Handling outliers in price: Figure 9 shows a price that features have serious data quality issues, caused by the extreme positive outliers (up to £9,999,999 and heavy right-skewed skewness, which an inaccurate value is £17,343 and a standard deviation of £46,437. To manage these outliers without throwing away useful data, winsorisation was chosen over other methods, such as removing and replacing. The method was applied to the 1st and 99th percentiles and, importantly, was stratified across the vehicle_condition categories (New and Used).

This stratification ensured outliers were capped relative to their own market segment, and the resulting summary confirmed the success: 1.99% of new cars and 1.95% of used cars were capped, guaranteeing neutral and equitable treatment across both segments. To ensure there are neutrals and reasonable treatment across both segments. After the winsorisation, the maximum price decreased from £9,999,999 to £ 96,201, and the standard deviation to £14,463, while the median and interquartile range remained stable.

The final distribution still shows a long upper whisker above 40,000; this means some high values are still far from the median but are no longer true outliers in a statistical sense. That is why 0.01% was used in the Winsorisation. This preserves realistic high-end price data relevant to luxury vehicles while effectively removing inaccurate extreme noise, therefore offering more stable and reliable central tendency without loss of statistical power.

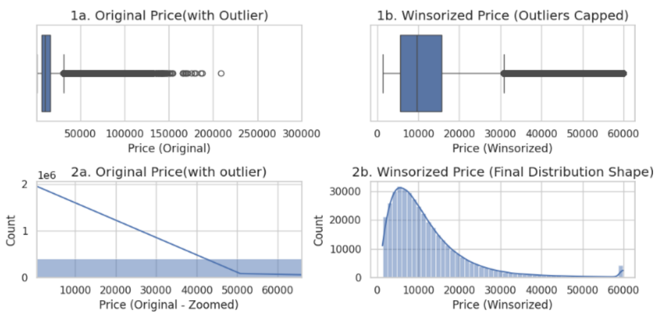

Figure 9: box plot for outlier and range, and histograms for shape and skewness comparison.

2.2. Feature Engineering, Data Transformations, Feature Selection

Feature engineering

First, some non-informative features like public reference and reg_code were dropped because no useful info there for the predictive signal, and they could introduce noise.

Some custom features were engineered using domain knowledge, helping to capture vehicle value drops, how they are used or market demand effects, like details that we cannot just pull from the original dataset.

The Vehicle_age feature was obtained from the year of registration, giving a clearer picture of how the value drops over time. Instead of using raw dates, this approach fits better with how prices change in the used car market. Mileage got log transformed as it was heavily skewed to the right when checked in data exploration. In this way, compressing extreme values and variation becomes stable, so high-mileage cars from disproportionately do not disproportionately influence the model's results. To differentiate usage patterns, we used mileage_per_year (total miles divided by how old the car is). This feature helps spot older cars barely driven and the ones already worn out. Also, an interaction feature, Age_x_Mileage, was added to show how age and usage combined, matching the uneven decrease in value seen when analysing price and age (Figure 5). These engineered features proved successful with strong predictive power and ranked highly in later feature importance analysis.

Model popularity features were also engineered; instead of treating the standard model as just a categorical label, we turn it into numeric features that indicate how popular each model is in the data. We counted how many times each model appears in the dataset. So, a higher count means a more common model and higher demand as well. Then combined model frequency and average selling price into one score, the score prioritises frequency because common models matter more. However, still considers price as expensive models may indicate desirability. To make it clearer and easier for encoding, we grouped models into three popularity classes (low, Medium, and High) because this helps avoid sparse metrics and supports ordinal encoding. This feature improved predictive power and added market context to the model beyond basic categorical encoding.

Data transformation

Since standard_make and Standard_model have many unique values and are high-cardinality categories, we encoded them using mean target encoding. We could have used one-hot encoding, but it results in sparse data because it creates too many columns, and there is a risk of overfitting.

For the popularity class, ordinal encoding was applied to assign numerical values (1,2,3) to preserve this ranking; this was chosen over one-hot encoding because the order would be lost. For low-cardinality nominal features like body_type and new_fuel_type, one-hot encoding was applied because they have few categories, and one-hot encoding works well as it creates a small number of columns and captures category-specific effects.

Numerical features got their missing values filled with the median imputation, and they were scaled using z-score, because scaling was a required method for KNN models to help them work better, also to stabilise linear models' training smoother. All the prepossessing steps were run through, and a ColumnTransformer pipeline was implemented to ensure consistency between training and test sets and no data leakage.

Feature selection

The feature selection process was used to choose only the most useful input features for the machine learning model. It helped improve accuracy, reduce overfitting, reduce noise, and made the model simpler and the results easier to understand. Before building the model, we dropped leakage and repeated columns. For example, the original price feature derived from variables dependent on the target and highly collinear attributes, such as year of registration, was swapped out for vehicle age.

For linear regression, Recursive Feature Elimination with Cross-Validation (RFECV) was applied to iteratively remove less important predictors and evaluate performance at each step. This process identified a smaller, more informative feature set that maintained predictive accuracy while improving model stability.

In contrast, tree-based models were trained using all available features because these algorithms inherently perform feature selection during the splitting process. Analysis of feature importance revealed that vehicle age, mileage, and popularity were the most influential predictors.

3. Model Building

3.1. Algorithm Selection, Model

Instantiation and Configuration

Three types of models were chosen: Linear Regression, K-nearest neighbours (KNN), and decision tree regression. Together, all these models represent increasing complexity, allowing us to compare performance across simple and complex approaches.

Linear regression was picked first as a baseline model because it is simple, computationally efficient, and interpretable. It assumes a straight-line relationship between features (independent variables) and the target value, making it a good starting point. We trained three different versions: first, a baseline linear regression without feature selection was trained to see how well it predicts once the data is cleaned up. Then, a selected KBest method with the F-statistic is used to keep the top 10 strongest features based on statistical tests. This reduces noise and dimensionality, so it can improve generalisation. Thirdly, Recursive feature elimination with cross-validation (RFECV) was used to select the best group of features by testing performance across folds. These three approaches allow us to compare no feature selection, fixed selection top 10 features, and adaptive selection (RFECV). This helps answer: do recursive features improve accuracy or efficiency, do adaptive methods outperform fixed selection and simple models sufficiently, or do complex models add value?

KNN regression was used to manage the patterns that are not strictly linear. Other models are assuming a specific formula, but KNN needs to compare every data point to its closest neighbours based on distance during prediction. All the numeric features and variables were standardised, and the categorical features were converted to numerical by encoding them. However, only 20% of the data was used as training data, to ensure the sample was stratified to preserve the price distribution by grouping prices into bins, because the dataset is large (over 400,000 rows), training all would take a long time and be expensive. The model was set up to 11 neighbours, weighted by distance, so closer points matter more. The ball tree algorithm was used to speed up the nearest neighbours' searches. This configuration achieved a balance between accuracy and computational efficiency.

The last model used was a Decision tree to capture the complex trends in vehicle pricing, for example, a sudden drop in value at a certain age and mileage point. Compared to KNN, decision trees do not need feature scaling and can manage non-linear relationships between features and outcomes effectively. First, we trained a basic tree with default settings to set a benchmark and check for overfitting by comparing training and testing results.

All models were built using pipelines, as preprocessing, feature selection and model training were combined into one workflow. The pipeline ensures all model gets the same transformation and prevents data leakage by applying preprocessing only on the training set.

3.2. Grid Search, and Model Ranking and

Selection

For linear regression models, fivefold cross-validation was evaluated to compare the baseline, SelectKBest and RFECV. The feature selection reduced dimensionality, improved interpretability and performance improvements were modest. Among them, RFECV stood out with the strongest linear performance, indicating that removing weak predictor features enhanced model stability; however, it could not fully capture nonlinear pricing patterns, limiting its predictive capability compared to more flexible algorithms.

The KNN model was evaluated using three-fold cross-validation on a stratified subsample of the dataset. Though it performed well during training and built competitive test accuracy, it showed signs of overfitting. KNN relies on comparing distances between data points; it requires a lot of memory and computation when the data is large. Thus, using a sample optimises computational efficiency, but it could also mean the model might not behave the same way on the full dataset. These constraints reduce how fit for deployment it is, despite its ability to capture nonlinear relationships effectively.

The last model, a decision tree made using GridSearchCV with three-fold cross-validation. Grid search identified the best tree hyperparameters (‘max_depth': 20, 'min_samples_leaf': 5, 'min_samples_split': 20) to control model complexity and decrease overfitting. Tuning the decision tree reached a good balance between bias and variance, significantly improved test performance, while reducing overfitting compared to the untuned baseline. The final tree model relates to the best estimator discovered through GridSearchCV and was refit on the full training set using the best hyperparameters.

Table 2 shows the model comparison summary that reports R2, MAE, RMSE, and cross-validated MAE for both testing and training sets. Table 2 displays that the tuned decision tree got the best overall performance with the highest test R2(0.949) and the lowest test MAE (1,788), suggesting strong generalisation. KNN achieved R2(0.937) but needed heavy subsampling and showed instability on the full dataset, making it less realistic. Linear regression performed less compared to the other models, with much lower R2(0.829) and the highest error values CV MAE Std (21.7), suggesting poor robustness.

Given its accuracy and consistent performance, the tuned decision tree was picked as the final model.

	Model	R2 Test	R2 Train	MAE Test	MAE Train	RMSE Test	RMSE Train	CV MAE Mean	CV MAE Std
0	Decision Tree (Tuned)	0.949	0.967	1787.89	1477.35	3227.40	2620.42	1850.56	9.70
1	kNN Regression (Subsampled)	0.937	0.995	1921.78	209.53	3605.93	1068.05	2003.36	12.12
2	Linear Regression (RFECV)	0.829	0.829	3785.15	3815.59	5937.99	5999.47	3816.90	21.74

Table 2: Model comparison across metrics.

4. Model Evaluation and Analysis

4.1. Coarse-Grained Evaluation/Analysis

Figure 10 shows visual comparisons of how all three models performed on the test set. The strongest one was a tuned decision tree with the highest accuracy R^2(0.949) and the lowest error MAE (£1,788), which denotes it simplifies well. KNN was the second strongest R^2 (0.937) but needed to use sampling to run efficiently; it showed weaker stability across folds. Linear regression scored a lower R^2(0.829) even after using RFECV to select features, and the largest error was MAE (£3,785). The feature selection helped the linear model a bit; however, the improvement was minor compared to on linear models.

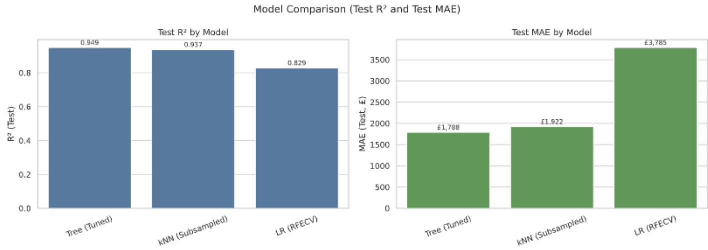

Figure 10: A bar chart showing how the three models performed on the test set.

4.2. Feature Importance

After completing the comparison of all the model performances, feature importance analysis was conducted for the tuned decision tree model. Figure 11 shows that the top nine features importance ranked by their strongest and contribution to predicting vehicle prices. The result of the bar chart indicates that the standard model was the most significant feature, which had a much higher importance score than all others. This finding makes sense as the specific models strongly define market value, indicating brand reputation demand. Vehicle_age and mileage_per_year were the second most key features, which align with depreciation trends, for example, older vehicles and those once driven more regularly are liable to lose value faster. Other features like model_popularity and the interaction term Age_x_Mileage also influenced meaningfully, suggesting the demand indicator and combined age usage effects influence pricing. Several others feature, such as standard_make, log_mileage and model_count, had quite low importance, indicating that while they add some predictive value, they are far less important than model identity and usage trends.

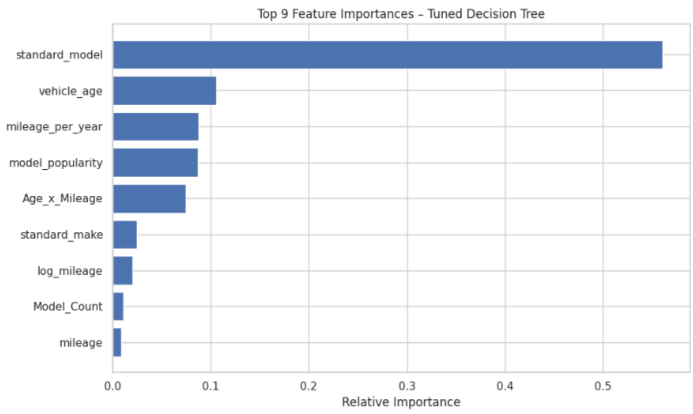

Figure 11: Features Importance

The cumulative importance plot in Figure 12 presents how predictive power is spread across features. The first feature alone explains more than half of the model's predictive power, and by the fourth feature, the total goes over 80%. After about six or seven features, the curve reaches 95% and flattens, meaning extra features add little. This indicates the decision tree depends mostly on a few key factors like model, age, and mileage. The other features have only a small predictive effect. As most of the predictions come from a small set of features, the standard model could work well even if we reduce the number of features.

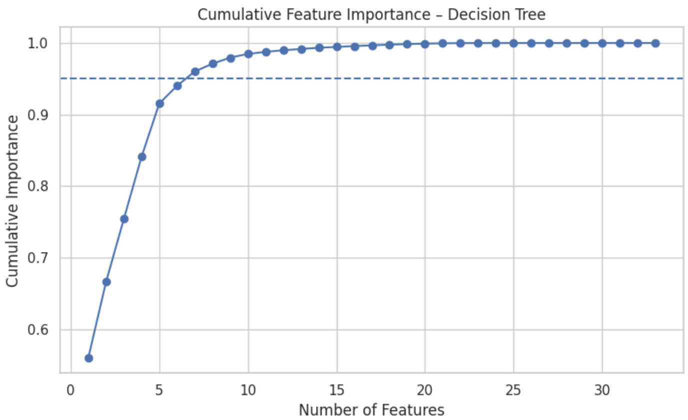

Figure 12: Cumulative feature importance

4.3. Fine-Grained Evaluation

Figure 13 shows residuals against the predicted price for the tuned decision tree. It clearly displays that most errors are centred around zero, suggesting good overall accuracy. The spread becomes wider for higher predicted prices, and the LOWESS trend line rises slightly at the top end, suggesting the model tends to underpredict for the most luxurious and high-performance cars. The figure displays a few outliers, but they do not stand out from the pattern. The model performs well across price ranges, with greater uncertainty at the luxury end.

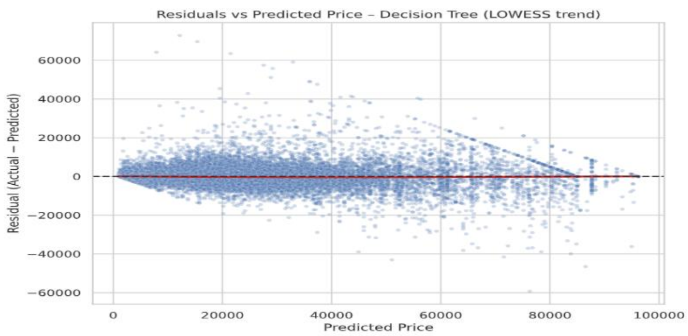

Figure 13: Residuals vs predicted price.

The histogram of residual (Figure 14) shows that the mean residual is close to zero, as it states -£10, indicating the tuned decision tree does not consistently overpredict or underpredict. Figure 14 also shows that most predication is within ±£3,000–£5,000 of the actual price, which is reasonable as given the dataset's size and price range. Looking at the shape of the distribution shows a bell-shaped distribution, indicating errors are random rather than systematic. It also shows a few outliers, but it is not a lot. These finding confirms that the decision tree model is well trained and performs steadily across most cases.

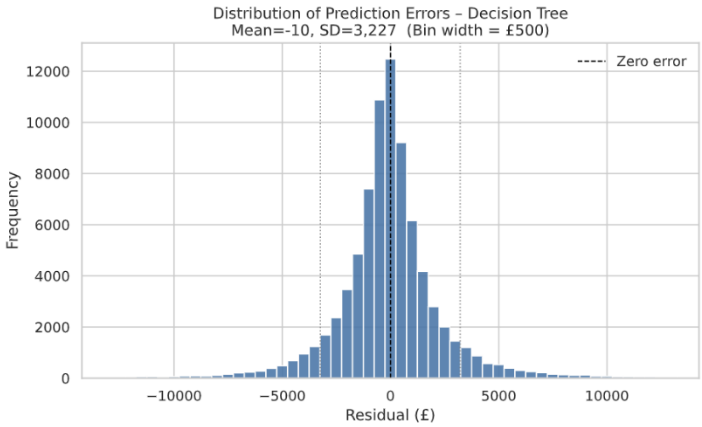

Figure 14: A histogram showing the residual distribution.

Figure 15 illustrates the relationship between absolute predication error and vehicle age. It clearly shows that most cars are under 20 years old, and their errors are focused below £10,000 and even some much lower. For older cars, above 30 years, the error becomes more scattered all over and rarely exceptionally large, for example, up to £70,000. This is due to high-performance and luxury cars like Ferrari, Porsche because of their unpredictable market values. This trend implies that the decision tree performs well for the typical age range but struggles with luxury and performance vehicles, where market values are challenging to predict.

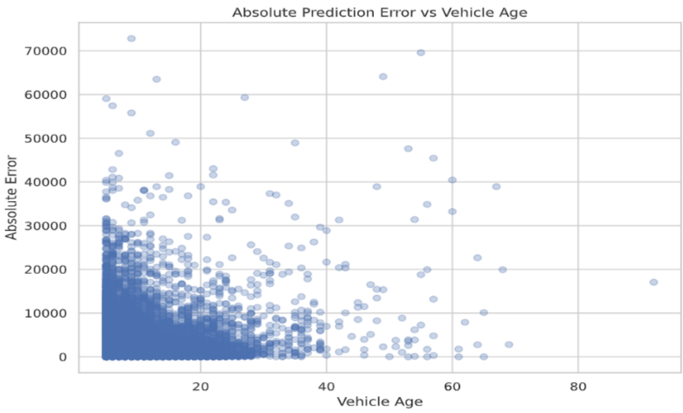

Figure 15: Absolute prediction error vs vehicle age

The violin plot in Figure 16 compares predication errors across five price segments. The segment starts from £1,000 to £96,000. Errors are smallest and consistent for budget-friendly price cars, typically within £2,000. Because as prices increase, the spread of errors also increases, with luxury and high-performance cars, indicating significantly higher variability and sometimes large errors. The pattern signals how challenging of predict high-end cars, where unique features and market reasons introduce greater uncertainty.

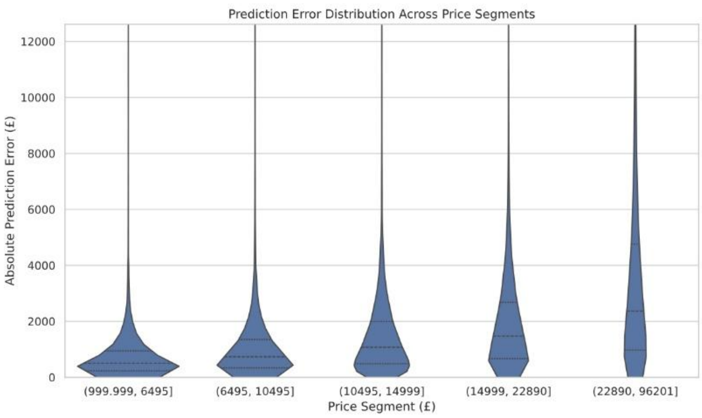

Figure 16: Violin plot showing prediction error distribution across price segments.

Figure 17 provides the final sanity check plot on model fit, showing that most points cluster along the diagonal, implying a strong fit. However, some dispersion shows at the highest price levels, proving earlier findings. The tuned decision tree demonstrates strong predictive performance. Minor deviation occurs at the highest price levels.

Figure 17: scatter plot showing predicted vs actual price for the turned decision tree.